PRACTICAL 8-A

Object Oriented Programming

AIM:

To implement and understand the concepts specified in Practical 8 of the CEUE203 Object Oriented Programming practical list, using Java. The work covers Part A: Guarded Calculator, Warehouse Exceptions, AutoCloseable and Part B: Bank exception hierarchy and transfer handling.

CODE SCREENSHOT:

GuardedCalculator.java 1

Part A > Practical 8-A > GuardedCalculator.java > ...

```java
import java.util.*;

class DivideByZeroException extends Exception {
    DivideByZeroException(String m) {
        super(m);
    }
}

public class GuardedCalculator {
    Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        while (true) {
            try {
                System.out.print("First number (q to quit): ");
                String x = sc.next();
                if (x.equalsIgnoreCase("q"))
                    break;
                double a = Double.parseDouble(x);
                System.out.print("Operator and second number: ");
                String op = sc.next();
                double b = Double.parseDouble(sc.next());
                double result;
                switch (op) {
                    case "+" -> result = a + b;
                    case "-" -> result = a - b;
                    case "*" -> result = a * b;
                    case "/" -> {
                        if (b == 0)
                            throw new DivideByZeroException(m: "Cannot divide by zero");
                        result = a / b;
                    }
                    default -> throw new IllegalArgumentException("Unknown operator");
                }
                System.out.println("Result: " + result);
                break;
            } catch (NumberFormatException e) {
                System.out.println("Invalid number");
            } catch (DivideByZeroException | IllegalArgumentException e) {
                System.out.println(e.getMessage());
            } finally {
                System.out.println("Attempt logged");
            }
        }
    }
}
```

Code Screenshot 1

ResourceDemo.java

Part A > Practical 8-A > ResourceDemo.java > ...

```java
class SampleResource implements AutoCloseable {
    public void use() {
        System.out.println("Using resource");
    }

    public void close() {
        System.out.println("Resource closed");
    }
}

public class ResourceDemo {
    Run | Debug
    public static void main(String[] args) {
        try (SampleResource r = new SampleResource()) {
            r.use();
            if (args.length > 0)
                throw new RuntimeException("Demo failure");
        } catch (RuntimeException e) {
            System.out.println("Reported: " + e.getMessage());
        }
    }
}

```

Code Screenshot 2

Warehouse.java

Part A > Practical 8-A > Warehouse.java > ...

```java
import java.util.*;

class OutOfStockException extends Exception {
    final int shortfall;

    OutOfStockException(int s) {
        super("Out of stock by " + s);
        shortfall = s;
    }
}

class InvalidQuantityException extends Exception {
    InvalidQuantityException() {
        super("Quantity must be positive");
    }
}

public class Warehouse {
    private final Map<String, Integer> stock = new HashMap<>(Map.of("pen", 5, "book", 2));

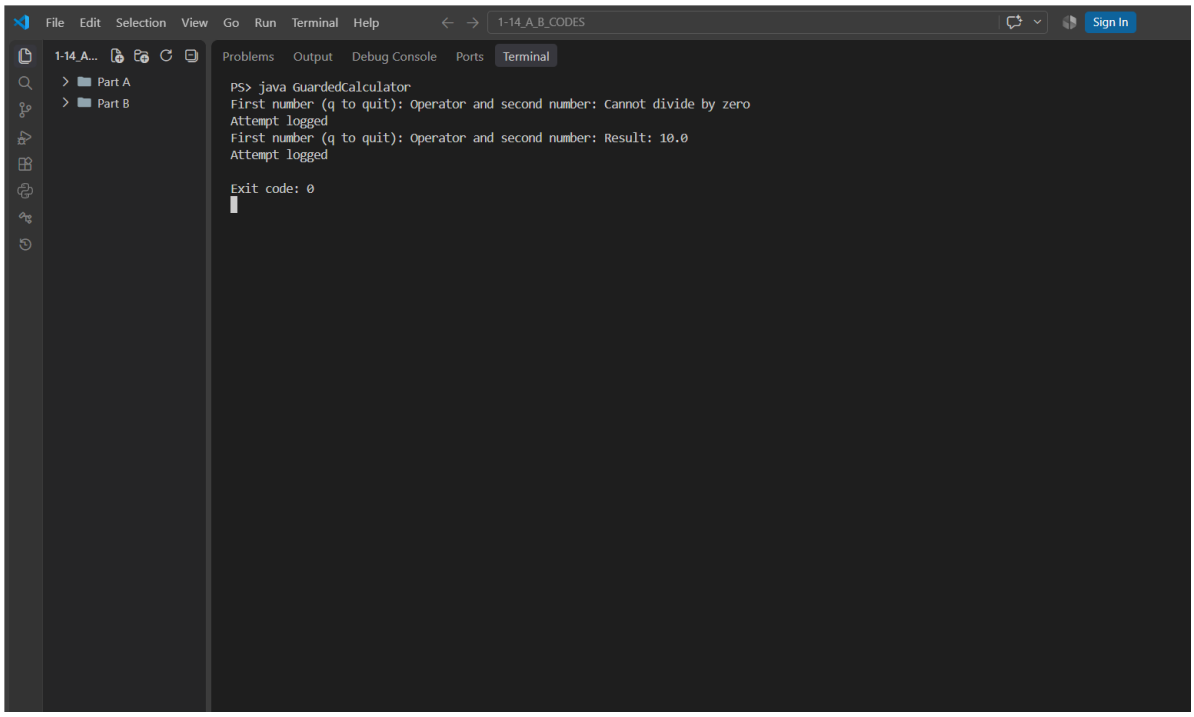
    public void issue(String item, int qty) throws OutOfStockException, InvalidQuantityException {
        if (qty <= 0)
            throw new InvalidQuantityException();
        int available = stock.getOrDefault(item, 0);
        if (qty > available)
            throw new OutOfStockException(qty - available);
        stock.put(item, available - qty);
        System.out.println("Issued " + qty + " " + item);
    }

    Run | Debug
    public static void main(String[] args) {
        Warehouse w = new Warehouse();
        Object[][] req = { { "pen", 2 }, { "book", 5 }, { "pen", 0 } };
        for (Object[] r : req)
            try {
                w.issue((String) r[0], (int) r[1]);
            } catch (Exception e) {
                System.out.println(e.getMessage());
            }
    }
}

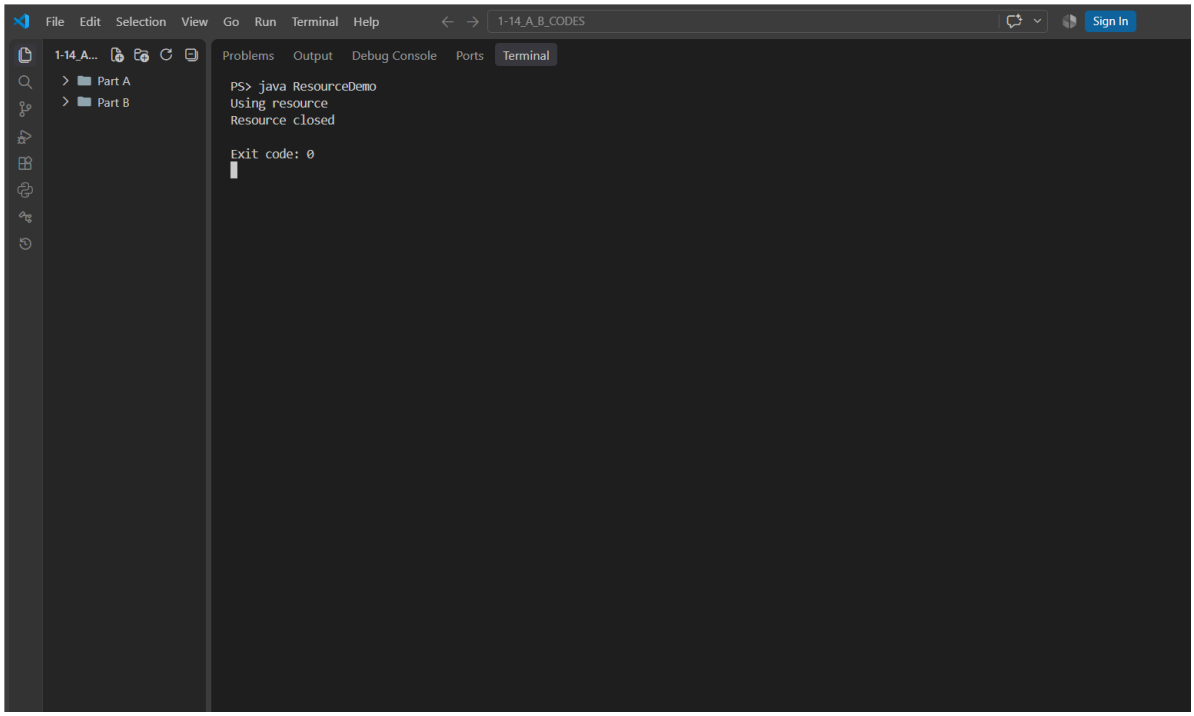
```

Code Screenshot 3

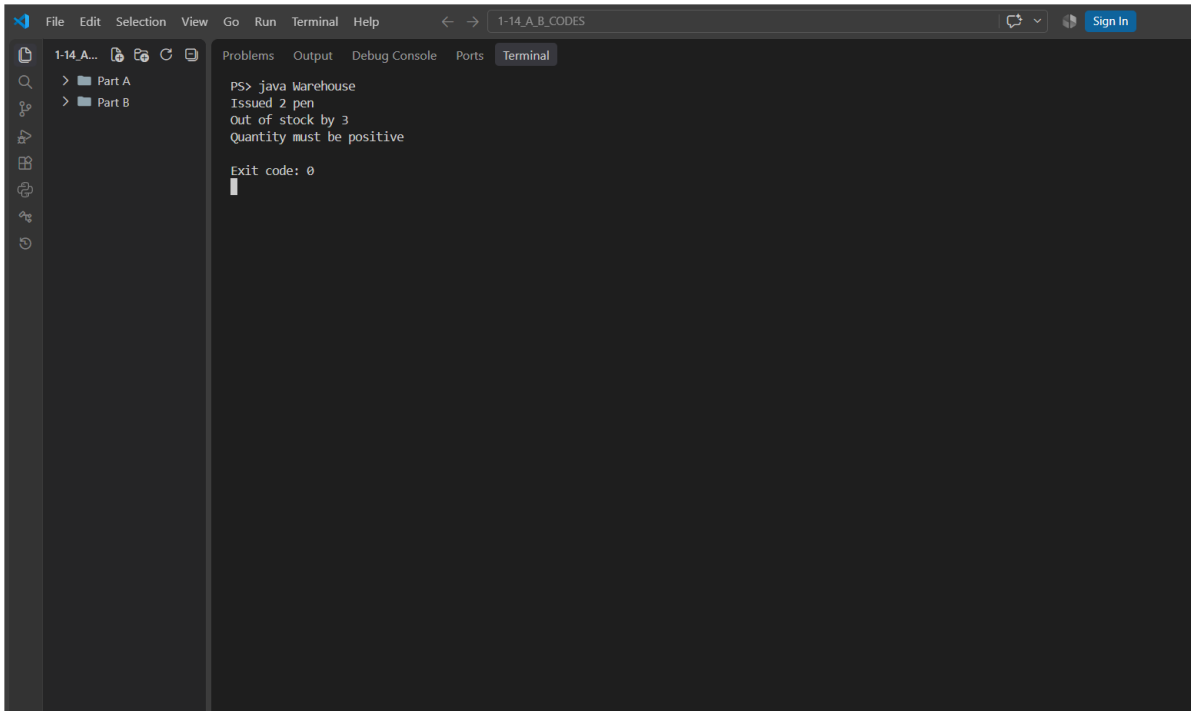
OUTPUT SCREENSHOT:



Output Screenshot 1



Output Screenshot 2



Output Screenshot 3

LEARNING OUTCOMES:

Handle invalid operations safely with custom exceptions, try-catch-finally, and try-with-resources.